

Getting-Started with HAPI

How do I use HAPI to find and get data?

1. Search for your data to see if any HAPI server has it
<https://hapi-server.org/search/>
Other Heliophysics search sites report HAPI access:
Heliophysics Data Portal <https://heliophysicsdata.gsfc.nasa.gov/>
Heliodata: <https://helio.data.nasa.gov/>
2. Use the HAPI Data Explorer to examine your dataset
<https://hapi-server.org/servers/>
This offers an intuitive, interactive way to explore data from known HAPI servers
3. The HAPI Data Explorer (link in step 2) can write a script showing how to obtain data.
This script is easily modifiable and can be added into your workflow.
Languages available: Python, IDL, R, julia, JavaScript, wget, curl

HAPI Tools for Scientists

- HAPI data explorer: <https://hapi-server.org/servers/>
- tutorials on Github: <https://github.com/heliophysicsPy/summer-school/tree/main/hapi-tutorial>
- videos from PyHC summer School on YouTube:
 - <https://www.youtube.com/playlist?list=PLDKhoNyHGTFZmalpqMNRFEf-hGFV-XhRk>
 - HAPI tutorial is on Day 1; HAPI-specific items start at 1:08:15 (1 hr, 8 min 15 sec in)
- reader libraries in multiple languages: Python, Java, IDL, Matlab, R
 - look for reader codes at the HAPI organization at Github: <https://github.com/hapi-server>
- many existing tools already can read from HAPI servers: SPEDAS / PySPEDAS, Autoplot
- KNMI Timeline Viewer visualization tool available: <https://spaceweather.knmi.nl/viewer/>
 - You can use this JavaScript visualizer for any HAPI server content
 - Open-source: <https://gitlab.com/KNMI-OSS/spaceweather/knmi-hapi-timeline-viewer>

HAPI Tools for Data Providers / Developers

- Well-written, comprehensive specification: <https://github.com/hapi-server/data-specification/>
- Over 30 open-source projects (clients servers, simple visualization, caching)
 - See HAPI Github organization: <https://github.com/hapi-server/>
- Verifier mechanism comprehensively checks if your server meets the spec
 - <https://hapi-server.org/verify/>
 - Can download and run the verifier locally or through an existing service
- Server implementations in multiple languages: Python, Java, Node.js
- Mailing list at hapi-dev@groups.io
- Developer's weekly meeting, currently Thu, 9am Eastern
See meeting invites at mailing list: <https://groups.io/g/hapi-dev/topics>

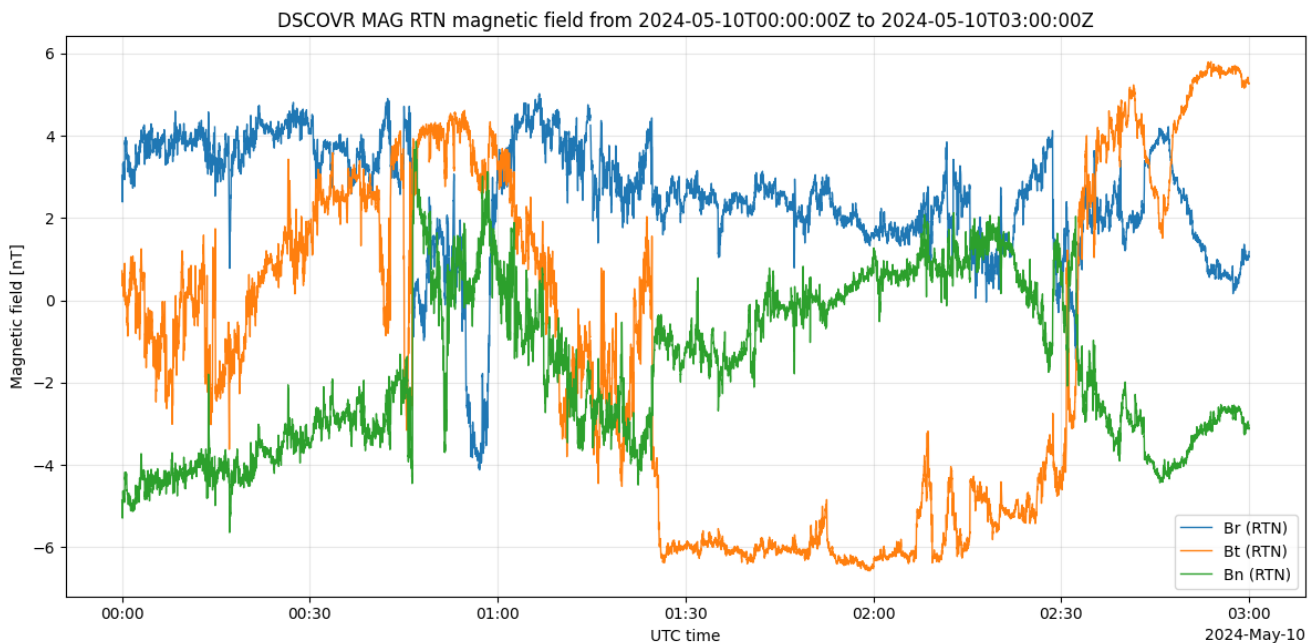
Easy-mode: getting AI to write your HAPI scripts!

HAPI is a public specification and is well-known to AI. It is fairly easy to get a chat bot or coding agent to use the existing HAPI client library to write an access script to get your data.

Sample prompt given to ChatGPT 5.5 Pro:

```
Write a python script to get some specific data from the CDAWeb HAPI server. Use the hapiclient library available at PyPI for accessing the HAPI server, and also make sure to use the already available routine hapiclient.hapitime.hapitime2datetime for parsing HAPI time values into Python datetime values. The data I want is from the NOAA Discover spacecraft and I want the vector magnetometer data. Write a python script that gets this data from CDAWeb using HAPI, and plots the 3 components of the magnetic field using matplotlib.
```

This prompt generated a script, and that script generated this plot:



The AI-generated script is below. On a command line, do the following to run this script:

```
% echo Creating Python environment for HAPI development
% python3 -m venv hapi_code
% cd hapi_code
% echo Assuming Bash shell, activate the Python virtual environment:
% source bin/activate
% echo Paste the code from the plot_dscover_mag_hapi.py into a file of that name.
% echo Install needed Python modules into this Python virtual environment
% python -m pip install hapiclient numpy matplotlib
% python plot_dscover_mag_hapi.py --start 2024-05-10T00:00:00Z --stop 2024-05-10T03:00:00Z --coord RTN
```

```
#!/usr/bin/env python3
```

```
"""
```

Fetch DSCOVR vector magnetometer data from CDAWeb through HAPI and plot the three magnetic-field components.

Install dependencies:

```
python -m pip install hapiclient numpy matplotlib
```

Example:

```
python plot_dscovr_mag_hapi.py \
    --start 2024-05-10T00:00:00Z \
    --stop 2024-05-10T03:00:00Z \
    --coord RTN
```

Use --coord GSE to plot Bx, By, Bz in GSE coordinates instead.

```
"""
```

```
import argparse
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
```

```
from hapiclient import hapi
from hapiclient.hapitime import hapitime2datetime
```

```
SERVER = "https://cdaweb.gsfc.nasa.gov/hapi"
DATASET = "DSCOVR_H0_MAG"
```

```
def get_parameter_meta(meta, name):
    """Return the metadata dictionary for a named HAPI parameter."""
    for parameter in meta.get("parameters", []):
        if parameter.get("name") == name:
            return parameter
    return {}
```

```
def fetch_dscovr_mag(start, stop, coord="RTN", mask_bad=True):
    """
```

Fetch DSCOVR MAG vector data from CDAWeb HAPI.

Parameters

start, stop : str

HAPI/ISO time strings, for example "2024-05-10T00:00:00Z".

```
coord : {"RTN", "GSE"}
```

Coordinate system to request. RTN uses B1RTN; GSE uses B1GSE.

```
mask_bad : bool
```

If True, also request FLAG1 and mask samples whose quality flag is not 0.

Returns

```
-----
```

```
times : list[datetime.datetime]
```

Python datetime values parsed with hapitime2datetime.

```
B : numpy.ndarray
```

Shape (N, 3), magnetic field components in nT.

```
labels : list[str]
```

Component labels for plotting.

```
units : str
```

Units string, usually "nT".

```
"""
```

```
coord = coord.upper()
```

```
if coord not in {"RTN", "GSE"}:
```

```
    raise ValueError("coord must be 'RTN' or 'GSE'")
```

```
vector_param = f"B1{coord}"
```

```
parameters = vector_param
```

```
if mask_bad:
```

```
    parameters += ",FLAG1"
```

```
data, meta = hapi(SERVER, DATASET, parameters, start, stop)
```

```
if len(data) == 0:
```

```
    raise RuntimeError(
```

```
        "The HAPI request returned no records. "
```

```
        "Try a different start/stop time range."
```

```
)
```

```
# Use the hapiclient-provided HAPI time parser, as requested.
```

```
times = hapitime2datetime(data["Time"])
```

```
# HAPI vector parameters come back as an N x 3 array.
```

```
B = np.asarray(data[vector_param], dtype=float)
```

```
if B.ndim != 2 or B.shape[1] != 3:
```

```
    raise RuntimeError(
```

```
        f"Expected {vector_param} to have shape (N, 3), got {B.shape}"
```

```
)
```

```
# Mask known fill values. The DSCOVR MAG fill value is very large negative
```

```
# in the metadata, so this threshold is safe relative to physical nT values.
```

```
B[B < -1.0e30] = np.nan
```

```

if mask_bad and "FLAG1" in data.dtype.names:
    # DSCOVR MAG FLAG1: 0 means good data.
    good = np.asarray(data["FLAG1"]) == 0
    B[~good, :] = np.nan

parameter_meta = get_parameter_meta(meta, vector_param)
units = parameter_meta.get("units", "nT")

if coord == "RTN":
    labels = ["Br (RTN)", "Bt (RTN)", "Bn (RTN)"]
else:
    labels = ["Bx (GSE)", "By (GSE)", "Bz (GSE)"]

return times, B, labels, units

def plot_components(times, B, labels, units, title):
    """Plot the three magnetic-field components."""
    fig, ax = plt.subplots(figsize=(12, 6))

    for i, label in enumerate(labels):
        ax.plot(times, B[:, i], linewidth=1.0, label=label)

    ax.set_title(title)
    ax.set_xlabel("UTC time")
    ax.set_ylabel(f"Magnetic field [{units}]")
    ax.grid(True, alpha=0.3)
    ax.legend(loc="best")

    locator = mdates.AutoDateLocator()
    ax.xaxis.set_major_locator(locator)
    ax.xaxis.set_major_formatter(mdates.ConciseDateFormatter(locator))

    fig.tight_layout()
    plt.show()

def main():
    parser = argparse.ArgumentParser(
        description="Plot DSCOVR vector magnetometer data from CDAWeb HAPI."
    )
    parser.add_argument(
        "--start",
        default="2024-05-10T00:00:00Z",
        help="Start time, for example 2024-05-10T00:00:00Z",
    )
    parser.add_argument(

```

```

    "--stop",
    default="2024-05-10T03:00:00Z",
    help="Stop time, for example 2024-05-10T03:00:00Z",
)
parser.add_argument(
    "--coord",
    choices=["RTN", "GSE", "rtn", "gse"],
    default="RTN",
    help="Coordinate system to plot: RTN uses B1RTN, GSE uses B1GSE.",
)
parser.add_argument(
    "--no-quality-mask",
    action="store_true",
    help="Do not use FLAG1 to mask bad/nonzero-quality samples.",
)

args = parser.parse_args()

times, B, labels, units = fetch_dscover_mag(
    start=args.start,
    stop=args.stop,
    coord=args.coord,
    mask_bad=not args.no_quality_mask,
)

title = (
    f"DSCOVER MAG {args.coord.upper()} magnetic field "
    f"from {args.start} to {args.stop}"
)

print(f"Fetches {len(times)} samples from {DATASET}.")
plot_components(times, B, labels, units, title)

if __name__ == "__main__":
    main()

```